



Figure 4: USB device overview

```

$ lsusb
Bus 001 Device 001: ID 0924:0202 Logitech Inc.
Bus 001 Device 002: ID 046d:08d0 Microsoft Corp.
Bus 001 Device 003: ID 05e3:0608 Genesys Logic, Inc.
Bus 001 Device 004: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 005: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 006: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 007: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 008: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 009: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 010: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 011: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 012: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 013: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 014: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 015: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 016: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 017: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 018: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 019: ID 04f2:b712 Chicony Electronics Co., Ltd.
Bus 001 Device 020: ID 04f2:b712 Chicony Electronics Co., Ltd.

```

Figure 5: Pen driver in action

```

return 0;
}

static void pen_disconnect(struct usb_interface *interface)
{
    printk(KERN_INFO "Pen drive removed\n");
}

static struct usb_device_id pen_table[] =
{
    { USB_DEVICE(0x058F, 0x6387) },
    {} /* Terminating entry */
};

MODULE_DEVICE_TABLE(usb, pen_table);

static struct usb_driver pen_driver =
{
    .name = "pen_driver",
    .id_table = pen_table,
    .probe = pen_probe,
    .disconnect = pen_disconnect,
};

static int __init pen_init(void)

```

```

{
    return usb_register(&pen_driver);
}

static void __exit pen_exit(void)
{
    usb_deregister(&pen_driver);
}

module_init(pen_init);
module_exit(pen_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Anil Kumar Pugalia <email@sarika-pugs.com>");
MODULE_DESCRIPTION("USB Pen Registration Driver");

```

Then, the usual steps for any Linux device driver may be repeated:

- Build the driver (.ko file) by running *make*.
- Load the driver using *insmod*.
- List the loaded modules using *lsmod*.
- Unload the driver using *rmmod*.

But surprisingly, the results wouldn't be as expected. Check *dmesg* and the *proc* window to see the various logs and details. This is not because a USB driver is different from a character driver—but there's a catch. Figure 3 shows that the pen drive has one interface (numbered 0), which is already associated with the usual *usb-storage* driver. Now, in order to get our driver associated with that interface, we need to unload the *usb-storage* driver (i.e., *rmmod usb-storage*) and re-plug the pen drive. Once that's done, the results would be as expected. Figure 5 shows a glimpse of the possible logs and a *proc* window snippet. Repeat hot-plugging in and hot-plugging out the pen drive to observe the *probe* and *disconnect* calls in action.

Summing up

"Finally! Something in action!" said a relieved Shweta. "But it seems like there are so many things (like the device ID table, probe, disconnect, etc), yet to be understood to get a complete USB device driver in place," she continued. "Yes, you are right. Let's take them up, one by one, with breaks," replied Pugs, taking a break himself. 

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at <http://profession.sarika-pugs.com>. He can be reached at email@sarika-pugs.com.